

Bootstrap

Framework para diseño Web responsivo



¿Qué es Bootstrap?

Bootstrap es **un framework de código abierto para el desarrollo Web front-end** que nos permite construir interfaces de forma más rápida utilizando componentes ya definidos.

Pero más allá de lo técnico... Bootstrap aparece para resolver un problema concreto:

- ▶ diseñar interfaces desde cero lleva tiempo
- ▶ mantener consistencia es difícil
- ▶ cada desarrollador puede implementar estilos distintos

Por lo que plantea la idea de **reutilizar soluciones ya construidas**.

¿Qué es Bootstrap? (Cont.)

Bootstrap **utiliza HTML, CSS y JavaScript** para proporcionar:

- ▶ estructura (grilla)
- ▶ componentes (botones, formularios, navegación)
- ▶ reglas de diseño consistentes



Bootstrap en contexto

Bootstrap fue creado en Twitter como una forma de estandarizar el desarrollo de interfaces.

Con el tiempo evolucionó hacia:

- ▶ enfoque **mobile-first**
- ▶ uso de **Flexbox**
- ▶ eliminación de dependencias como jQuery



¿Por qué usar Bootstrap?

Bootstrap resuelve varios problemas comunes en el desarrollo Web:

- ▶ **Organización del diseño:** Permite estructurar la interfaz mediante un sistema de grilla adaptable.
- ▶ **Reutilización:** Ofrece componentes listos para usar: botones, formularios, navegación, etc.
Esto evita construir todo desde cero.
- ▶ **Consistencia:** Asegura que la interfaz tenga un comportamiento uniforme en distintos dispositivos.
- ▶ **Ahorro de tiempo:** Reduce la cantidad de CSS necesario.
- ▶ **Accesibilidad integrada:** Muchos componentes incluyen soporte para lectores de pantalla y navegación por teclado.

Diseño Mobile-First

Bootstrap sigue un enfoque mobile-first.

Esto significa que el **diseño se construye primero para pantallas pequeñas y luego se adapta a pantallas más grandes.**

Ahora bien, ¿por qué?

Porque hoy:

- ▶ gran parte del tráfico Web es móvil
- ▶ diseñar para pantallas pequeñas obliga a simplificar

Diseñar primero para lo limitado ayuda a tomar mejores decisiones.



Sistema de Grilla

Bootstrap organiza la pantalla en **12 columnas**.

Esto permite distribuir el contenido de **forma flexible**.

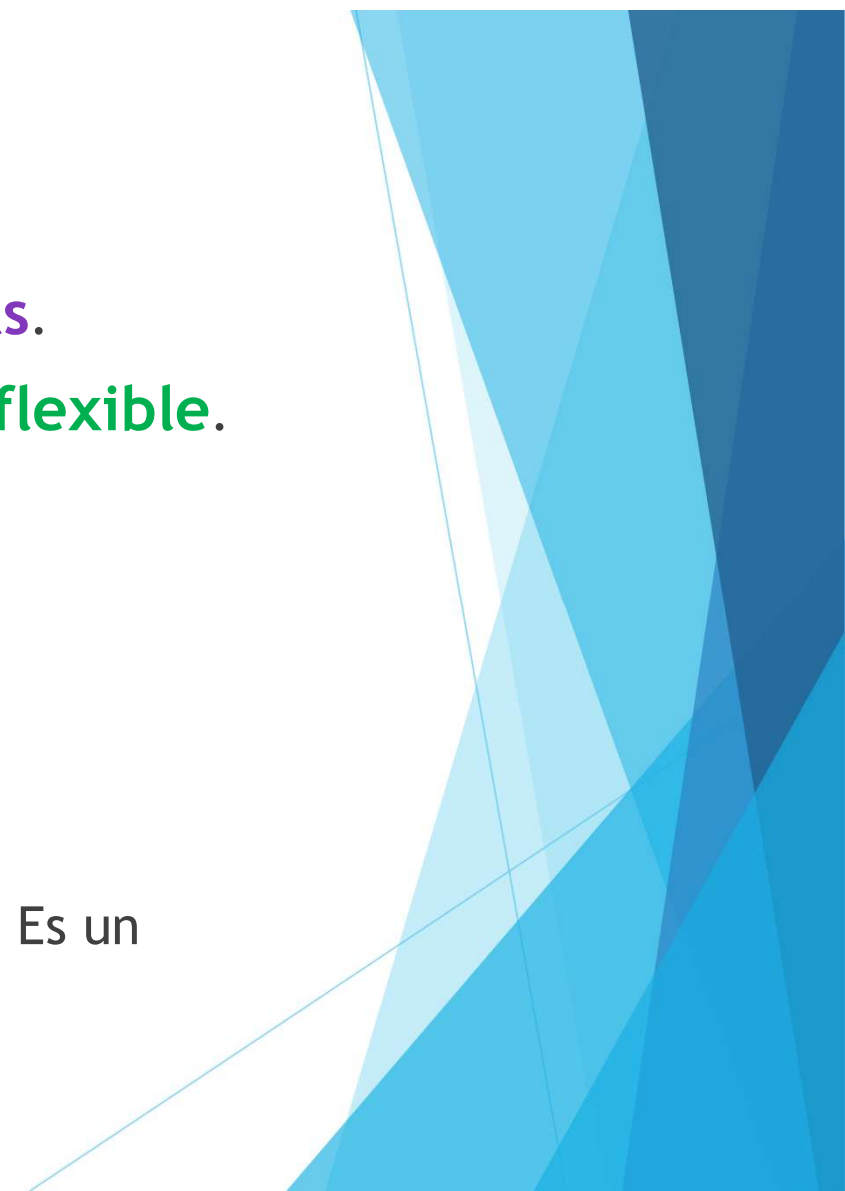
Por ejemplo:

- ▶ 2 columnas $\rightarrow 6 + 6$
- ▶ 3 columnas $\rightarrow 4 + 4 + 4$
- ▶ 4 columnas $\rightarrow 3 + 3 + 3 + 3$

¿Por qué 12?

Porque permite muchas combinaciones simples. Es un equilibrio entre:

- ▶ flexibilidad
- ▶ simplicidad



Sistema de Grilla (Cont.)

Los elementos principales son:

- ▶ **container:** delimita el espacio principal del contenido
- ▶ **row:** organiza filas agrupando las columnas
- ▶ **col:** define columnas para controlar el ancho y comportamiento del contenido

La grilla no es solo visual... es una forma de pensar la estructura de la interfaz.



Breakpoints

Permiten **adaptar el diseño según el tamaño de pantalla**. Esto significa que:

- ▶ la interfaz no es fija
- ▶ cambia según el contexto del usuario

Por ejemplo:

```
col-12 col-md-6
```

- ▶ ocupa todo en móvil
- ▶ la mitad en pantallas medianas (tablets)

El cliente (navegador) participa activamente en cómo se presenta la información.

Breakpoints (Cont.)

Breakpoint	Dimensiones	Clase	Descripción
xs (extra small)	<576px	.col-	Este es el tamaño predeterminado y no se define un breakpoint ya que el grid es mobile-first.
sm (small)	≥576px	.col-sm-	Para dispositivos móviles.
md (medium)	≥768px	.col-md-	Para tablets.
lg (large)	≥992px	.col-lg-	Para aplicaciones de escritorio.
xl (extra large)	≥1200px	.col-xl-	Para pantallas grandes
xxl (extra extra large)	≥1400px	.col-xxl-	Introducido en Bootstrap 5, para pantallas más grandes aún.

Contenedores

Son la base del layout en Bootstrap.

Permiten:

- ▶ centrar contenido
- ▶ controlar el ancho
- ▶ organizar la página

Existen tres tipos:

- ▶ **container:** Ancho fijo que cambia según el breakpoint. Clase por defecto.
- ▶ **container-fluid:** Siempre ocupa el 100% del ancho disponible.
- ▶ **container-{breakpoint}:** Fluido hasta un cierto tamaño, luego fijo.

Contenedores (Cont.)

Los contenedores responsive permiten especificar una clase que tiene un **100% de ancho** hasta que se alcanza el breakpoint especificado, después de lo cual se aplica **max-width** para cada uno de los breakpoints más altos.

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	X-Large ≥1200px	XX-Large ≥1400px
.container	100%	540px	720px	960px	1140px	1320px
.container-sm	100%	540px	720px	960px	1140px	1320px
.container-md	100%	100%	720px	960px	1140px	1320px
.container-lg	100%	100%	100%	960px	1140px	1320px
.container-xl	100%	100%	100%	100%	1140px	1320px
.container-xxl	100%	100%	100%	100%	100%	1320px
.container-fluid	100%	100%	100%	100%	100%	100%

Filas y columnas

Las filas organizan el contenido horizontalmente.

Las columnas definen cuánto espacio ocupa cada elemento.

```
<div class="container">
  <div class="row">
    <div class="col-12 col-md-6">Columna A</div>
    <div class="col-12 col-md-6">Columna B</div>
  </div>
</div>
```

Regla importante: **la suma de columnas no debe superar 12.**

Offset y alineación

El desplazamiento (offset) permite dejar columnas vacías a la izquierda de un elemento.

```
<div class="row">  
  <div class="col-12 col-md-4 offset-md-4">  
    Columna centrada  
  </div>  
</div>
```

También se puede usar:

- ▶ .ms-auto (margin-start auto) para alinear a la derecha.
- ▶ .me-auto (margin-end auto) para alinear a la izquierda.

Componentes comunes

Bootstrap incluye muchos componentes listos para usar:

- ▶ **Botones:** .btn, .btn-primary, .btn-outline-secondary, etc.

```
<button class="btn btn-success">Aceptar</button>
```

- ▶ **Navegación:** Navegación adaptable con menús desplegables y colapsables.
- ▶ **Tarjetas:** Bloques de contenido con borde, sombra y estructura definida.
- ▶ **Modales:** Ventanas emergentes para diálogos o formularios.
- ▶ **Formularios:** Estilos para inputs, selects, checks, validación, etc.

Todos los componentes se pueden personalizar o combinar con clases utilitarias.

Incorporación al HTML

Bootstrap se agrega al HTML mediante:

- ▶ CSS (estructura y estilos)
- ▶ JS (comportamiento de componentes)

Puede usarse:

- ▶ desde CDN (Content Delivery Network)
- ▶ localmente

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/  
dist/css/bootstrap.min.css" rel="stylesheet"  
crossorigin="anonymous">
```

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/  
dist/js/bootstrap.bundle.min.js"  
crossorigin="anonymous"></script>
```

Buenas prácticas

- ▶ Usar clases utilitarias (márgenes, colores, tamaños, alineaciones, etc.) para:
 - ▶ Acelerar el desarrollo.
 - ▶ Mantener el HTML más legible y directo.
 - ▶ Reducir la necesidad de crear hojas de estilo adicionales.
 - ▶ Favorecer la coherencia visual en el diseño.
- ▶ Evitar el anidamiento excesivo de columnas y filas.
- ▶ Reutilizar componentes en lugar de duplicar código.
- ▶ Separar lógicamente el HTML de las personalizaciones (usar archivos CSS propios si es necesario).
- ▶ Probar los diseños en distintos dispositivos o simuladores.

¿Qué estamos ganando... y qué estamos perdiendo?

Cuando usamos Bootstrap:

- ▶ desarrollamos más rápido
- ▶ logramos consistencia
- ▶ reducimos esfuerzo

Pero:

- ▶ perdemos control fino
- ▶ dependemos del framework
- ▶ muchas interfaces se parecen entre sí

**Usar un framework no elimina la complejidad...
cambia dónde la gestionamos.**



